Semana 5 · Capa de enlace confiable (sin TLS)

Fase de la propuesta: capa de enlace y TLS clásico (semanas 5-6).

Objetivo	Poner en marcha la capa de enlace confiable (tramas, CRC, confirmaciones y retransmisión) entre el ESP32 y la Raspberry, todavía sin TLS, y comprobar que no pierde ni corrompe datos.
Dedicación estimada	4–6 horas.
Antes de empezar	FRAME_SIZE decidido (semana 4).
Al terminar tendrán	50 corridas sin TLS sin errores y la comprensión del formato de trama (lo necesitarán en el capítulo de diseño).

Cómo funciona la capa de enlace

I2C no garantiza que los datos lleguen completos y en orden, y TLS lo exige. Por eso cada trama lleva un número de secuencia, la confirmación (ACK) de lo último recibido y un CRC-16. La Raspberry (maestro) hace **intercambios**: escribe una trama y luego lee la respuesta del ESP32 cuando este levanta READY. Si una trama no se confirma, se reenvía (ARQ de parada y espera).

MAGIC	TIPO	SEQ	ACK	LEN	CARGA ÚTIL (FRAME_SIZE − 8)	CRC-16
1 B	1 B	1 B	1 B	2 B	hasta 120 B (trama 128)	2 B

Campo	Uso
MAGIC (0xA5)	detecta tramas basura
TIPO	0 sondeo (sin datos), 1 datos, 2 control (órdenes y reportes), 3 reinicio
SEQ / ACK	número de la trama enviada / última trama del otro extremo recibida bien
LEN	bytes útiles en la carga
CRC-16/CCITT	detecta errores accidentales del bus (no protege contra un atacante: eso lo hace TLS)

Paso 1. Grabar el firmware del nodo (modo normal)

En la laptop: `git pull` (para tener `certs/servidor.pem`), regenerar el header del certificado y, en `main/CMakeLists.txt`, poner `set(MODO_ECO 0)` y su `FRAME_SIZE`:

```
cd ~/esp/proyecto && git pull
python herramientas/cert_a_header.py
cd esp32/firmware && idf.py -p PUERTO build flash
```

Pasen el USB a la Raspberry y abran la consola: debe decir `wolfSSL 5.9.4 (se requiere >= 5.9.4), FRAME_SIZE=<F>` y `Nodo TLS listo`. Cierren la consola (Ctrl+]) antes de seguir.

Paso 2. Primera corrida sin TLS

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 servidor_tls.py --grupo sin_tls --frame F --n 20 --bulk 0     # F = su FRAME_SIZE
```

Esperado: `"error": ""`, `"esp_estado": 0`, `"enl_errores_crc": 0`. Aquí la telemetría viaja **en claro**; es la condición de control de la semana 8.

Paso 3. Prueba de resistencia: 50 corridas

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 campana.py --bloque prueba_enlace --grupos sin_tls --tamanos 32,512 \
                   --reps 25 --reposo 10 --pausa 0.5 --frame F
python3 resumen_rapido.py
```

`campana.py` también arranca el registro del INA219: así prueban ambas cosas juntas. Esperado: 50 corridas correctas, contadores del enlace en 0 (o muy bajos).

Paso 4. (Opcional) Comprobar que la retransmisión funciona

Durante una corrida larga (`--n 200`), desconecten el cable READY unos 2 segundos y vuelvan a conectarlo. La corrida debe terminar sin error y `enl_timeouts_ready` / `enl_retransmisiones` deben ser mayores que 0: el enlace se recuperó solo.

Paso 5. Eficiencia del enlace

Con el JSON de una corrida, calculen `eficiencia = (enl_payload_tx + enl_payload_rx) / enl_bytes_bus`. Es la fracción del bus que lleva datos útiles (el resto son cabeceras, CRC y sondeos). Anótenla: sirve para explicar después por qué el handshake híbrido tarda lo que tarda.

Criterio de cierre de la semana

- ☐ El nodo arranca mostrando wolfSSL 5.9.4 y el FRAME_SIZE correcto.
- ☐ Una corrida sin_tls termina sin errores.
- ☐ 50 corridas de prueba_enlace correctas (resumen_rapido.py).
- ☐ Eficiencia del enlace calculada y anotada.
- ☐ Commit y push (incluyendo los archivos de data/raw/ de esta prueba).